CATILLION Timéo BTS CIEL 1

Mini rapport : détection de spams par réseau de neurones

1. Contexte

Beaucoup d'entreprises font de la publicité par e-mail, et on reçoit chaque jour des messages dont on ne veut pas. Il est long de les trier à la main. Le but : faire dire à un programme si un email est un spam ou un bon message. J'utilise les données Spambase.

2. Données

4601 emails. Chaque e-mail est décrit par 57 chiffres (combien de fois certains mots ou signes apparaissent, nombre de majuscules) et une colonne « spam » (1 = spam, 0 = bon message). Il y a 2 classes : 2 788 bons messages et 1 813 spams. Aperçu avec head() :

	word_freq_make	word_freq_address	word_freq_all	word_freq_3d	word_freq_our	capital_run_length_average	capital_run_length_longest	capital_run_length_total	spam
0	0.0	0.64	0.64	0.0	0.32	3.7560000000000002	61	278	1
1	0.21	0.28	0.5	0.0	0.14	5.114	101	1028	1
2	0.06	0.0	0.71	0.0	1.23	9.821	485	2259	1
3	0.0	0.0	0.0	0.0	0.63	3.537	40	191	1
4	0.0	0.0	0.0	0.0	0.63	3.537	40	191	1

3. Prétraitement

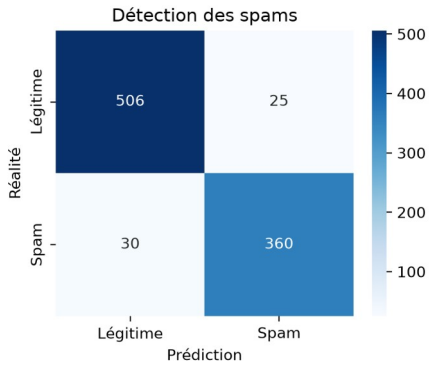
J'ai séparé les données : 80 % (3680 emails) pour entraîner le modèle et 20 % (921 emails) pour tester. J'ai mis toutes les colonnes à la même échelle avec StandardScaler. L'échelle est calculée sur les données d'entraînement seulement. Il n'y avait rien à nettoyer.

4. Modèle

Un réseau de neurones (MLPClassifier) : 57 entrées, deux couches de 16 puis 8 neurones, et 1 sortie (spam ou non). Un petit réseau suffit pour 57 chiffres.

5. Résultats

Sur le test, le modèle a bon dans 94,0 % des cas pour un entraînement (93,8 % en moyenne sur 5). Voici la matrice de confusion :



Sur 531 bons messages, 25 sont classés spam par erreur. Sur 390 spams, 30 ne sont pas détectés comme spam. Il y a à peu près autant des deux erreurs, mais bloquer un bon message est plus gênant.

6. Fiche de recette

Modèle testé : le réseau de neurones du fichier TP6.py (2 couches de 16 et 8 neurones). J'ai choisi les critères avant les tests. Pour un filtre anti spams, il faut bloquer le moins possible de bons messages et trouver le plus possible les spams.

Test : 921 emails (20 % des données) que le modèle n'a jamais vus. Le résultat change un peu à chaque entraînement, donc je le refais 5 fois et je compare aux seuils.

N°	Critère	Seuil	Résultat obtenu	Verdict
1	Bonnes réponses sur le test (moyenne des 5 essais)	≥ 92 %	93,79 % (min 93,38 %)	OK
2	Spams trouvés (moyenne)	≥ 90 %	91,28 % (min 89,49 %)	OK
3	Quand il dit « spam », c'est vrai (moyenne)	≥ 92 %	93,90 % (min 92,95 %)	OK
4	Bons messages classés spam par erreur (moyenne)	≤ 5 %	4,37 % (max 5,08 %)	OK
5	Écart entre entraînement et test	≤ 7 pts	5,4 pts (99,2 % / 93,8 %)	OK
6	Écart entre les 5 essais	≤ 1 pt	0,29 pt	OK
7	Tours d'entraînement avant la fin	< 1000	387 à 557	OK

Verdict : modèle accepté, avec des réserves. Les 7 critères sont respectés en moyenne et les résultats sont stables : le modèle trouve environ 91 % des spams et bloque moins de 5 % des bons messages. Réserves : on est proche des seuils sur le test 2 et 4, le modèle est meilleur sur l'entraînement (99 %) que sur le test (94 %), et environ 1 bon message sur 20 est bloqué, ce qui est un peu trop pour un vrai usage.

7. Bilan

J'ai appris à faire un projet complet : regarder les données, les préparer et entraîner un réseau de neurones. Difficulté : le résultat change à chaque entraînement, donc je l'ai refait 5 fois. Piste pour s'améliorer : empêcher le modèle d'apprendre trop par cœur (early_stopping) et le régler pour qu'il bloque moins de bons messages.